软件系统综合课程设计——练习 1

实验名称	系统功能测试			**成绩**	
姓 名	程泽凯	**学号**	2023112621	**专业班级**	计算机 2023-04 班
团队	11	项目名称		交大论坛	
实验地点	软件工程实验室（X7508）			**日 期**	2026 年 6 月 9 日

实验目的	（1）巩固所学到的黑盒测试方法。 （2）掌握测试工具、提高实际的测试能力。 （3）针对被测试项目进行功能测试和 WEBUI 测试，发现其存在的缺陷。
实验前提	（1）理解功能测试分析思路。 （2）熟悉测试用例设计测试方法。 （3）部署测试环境 （Junit、Selenium+Webdriver）
实验收获	通过本次实验，我进一步掌握了黑盒测试、测试用例设计和 Selenium 自动化测试方法，理解了功能验证与缺陷记录的重要性。

实验过程：

（1） 项目小组讨论，分析小组开发的系统，确定测试的范围，列出测试项（功能点）。

报告： 项目名称，简介，功能点

（2）按功能点分工，如有 12 功能点，则每组 4 人，每个学生分到 3 个功能点。

报告：用表格列出自己的测试功能点

（3）选择测试方法：基于黑盒测试方法中等价类+边界分析法判定表法，每个学生根据情况设计 20 个或更多的测试用例，这些用例要求相对关键，对功能点的验证有效，或发现 Bug 的可能性更高。

报告：针对每个功能点，选择不同的测试方法，逐步分析，完成测试用例。并通过列表展示

（4）每个学生向本组其他学生讲解自己是如何设计这些测试用例的，其他学生提出意见，然后大家讨论，最后修改、完善之前写的测试用例

（5）根据自己判断，决定在 IE、Chrome 执行哪些测试用例。小组同学可以交叉测试。

（6）记录发现的缺陷。

（7）基于 Selenium+Webdriver，将之前设计的测试用例转化为测试脚本，如 JAVA 的脚本，在 Eclipse 的 JUnit 框架上执行和调试这些脚本。

报告： 脚本

（8）最后写出一个完整的测试报告，包括分析思路、功能点清单、如何设计测试用例，脚本开发遇到哪些问题、测试环境、测试结果、Bug 列表、其他的体会感想

实验结果

作业提交内容：测试用例、测试脚本、以及完整的测试报告。

目录

一、 项目名称、简介与个人测试范围 1

二、 实验过程记录 1

2.1 小组讨论与测试范围确认 1

2.2 个人分工与交叉复核 1

2.3 用例评审后的调整 1

三、 个人负责功能点清单 1

四、 分析思路与测试方法 2

五、 功能点逐项详细分析 2

5.1 积分商城：积分兑换奖品 2

5.2 通知中心：查看通知 2

5.3 关注：关注/取消关注用户 2

5.4 后台管理：奖品管理 3

六、 测试用例列表（20 条） 3

七、 测试环境与浏览器执行选择 4

八、 测试执行过程与截图 4

8.1 截图流程 1：积分商城奖品列表 4

8.2 截图流程 2：积分兑换结果区域 5

8.3 截图流程 3：通知中心展开状态 5

8.4 截图流程 4：关注前的帖子详情页 6

8.5 截图流程 5：关注用户后的详情页 6

8.6 截图流程 6：取消关注后的详情页 7

8.7 截图流程 7：管理员账号登录状态 7

8.8 截图流程 8：后台奖品管理列表 8

8.9 截图流程 9：后台新增奖品结果 8

8.10 截图流程 10：后台刷新后的奖品列表 9

九、 脚本开发、执行与调试 9

十、 测试结果分析 9
十一、 Bug 列表 10
十二、 体会与收获 10
A 自动化测试脚本核心代码 10

一、 项目名称、简介与个人测试范围

项目名称为“西南交通大学校园论坛系统”。系统提供注册登录、帖子浏览、内容发布、互动、个人中心、积分成长、通知和后台管理等能力。本报告围绕本人的个人分工展开，重点记录测试分析、用例设计、执行结果和个人体会。

二、 实验过程记录

2.1 小组讨论与测试范围确认

本次测试开始前，小组先一起梳理论坛系统的主要功能，包括用户入口、内容浏览、内容发布、互动关系、个人资料、积分成长、通知和后台管理。讨论时没有简单按页面数量划分，而是把能够独立输入、独立观察输出、独立判定结果的功能作为测试项。本次个人分工为序号 13–16，因此本报告只展开本人负责的 4 个功能点。

2.2 个人分工与交叉复核

本人先根据负责范围写出用例初稿，然后检查每条用例是否有测试数据、操作步骤、预期结果和实际结果。之后小组成员之间进行交叉复核，主要看用例是否重复、是否能执行、结果是否能判断。页面类用例要求能在 Chrome 中找到对应入口；接口类用例要求能根据状态码、返回字段或刷新后的页面状态判断结果。

2.3 用例评审后的调整

评审后，本人主要调整了三类内容。第一类是只写“点击按钮”的用例，补充了点击后的页面文本、按钮状态或接口返回。第二类是只考虑正常流程的用例，补充了边界值和异常等价类。第三类是结果不容易复核的用例，改成能够通过页面元素、状态码或截图直接确认的场景。调整后，本人负责范围内形成 20 条用例。

三、 个人负责功能点清单

序号	一级功能	二级功能	用例数
13	积分商城	积分兑换奖品	5
14	通知中心	查看通知	5
15	关注	关注/取消关注用户	5
16	后台管理	奖品管理	5

四、 分析思路与测试方法

本人负责积分商城、通知中心、关注关系和新增的后台奖品管理。前三个功能直接影响用户互动闭环，后台奖品管理则影响商城数据来源，因此用例同时覆盖普通用户路径和管理员路径。

本次测试选择等价类划分、边界值分析和判定表法，是因为本人负责的功能既有输入框，也有登录态和状态变化。等价类可以减少重复输入，只选有代表性的合法和非法数据；边界值适合检查长度、分页、积分、库存等临界情况；判定表则适合整理“是否登录、资源是否存在、当前是否已经操作过”等组合条件。这样设计出来的用例更容易执行，也更容易说明为什么通过或失败。

五、 功能点逐项详细分析

5.1 积分商城：积分兑换奖品

积分商城测试先看奖品列表是否能加载，再看兑换条件是否正确。本人重点检查积分不足提示、兑换记录和刷新后的页面状态。

本功能点共 5 条用例，使用的方法包括有效等价类、判定表、状态迁移、UI 回归。执行时先准备登录态或测试数据，再按步骤操作，最后根据页面反馈、状态码或刷新后的页面状态判断。代表性执行结果为：奖品列表状态码 200，数量 2。本功能点通过 5 条，失败 0 条。

本功能点未记录失败用例，但仍保留边界和权限类用例，便于后续回归时继续复核。

5.2 通知中心：查看通知

通知中心不能只检查节点是否存在。本人点击通知入口后，会继续检查面板是否真正可见，通知内容能否读到，以及未读状态是否正常。

本功能点共 5 条用例，使用的方法包括 UI 回归、有效等价类、状态迁移、移动端 UI。执行时先准备登录态或测试数据，再按步骤操作，最后根据页面反馈、状态码或刷新后的页面状态判断。代表性执行结果为：通知面板 hidden=false 且 display=flex。本功能点通过 5 条，失败 0 条。

本功能点未记录失败用例，但仍保留边界和权限类用例，便于后续回归时继续复核。

5.3 关注：关注/取消关注用户

关注关系需要看前后状态。本人先关注作者，再取消关注，确认按钮状态和接口结果能对应起来。

本功能点共 5 条用例，使用的方法包括有效等价类、状态迁移、判定表、UI 验证。执行时先准备登录态或测试数据，再按步骤操作，最后根据页面反馈、状态码或刷新后的页面状态判断。代表性执行结果为：关注作者接口状态码 200。本功能点通过 5 条，失败 0 条。

本功能点未记录失败用例，但仍保留边界和权限类用例，便于后续回归时继续复核。

5.4 后台管理：奖品管理

后台奖品管理是新增的第 16 个功能点。本人把它作为管理员路径测试，重点看管理员能否查看奖品列表、新增奖品、刷新后记录是否仍然存在，以及普通用户是否会被权限拦截。

本功能点共 5 条用例，使用的方法包括有效等价类、边界值、判定表、状态迁移。执行时先准备登录态或测试数据，再按步骤操作，最后根据页面反馈、状态码或刷新后的页面状态判断。代表性执行结果为：后台奖品列表能够正常展示。本功能点通过 5 条，失败 0 条。

本功能点未记录失败用例，但仍保留边界和权限类用例，便于后续回归时继续复核。

六、 测试用例列表（20 条）

编号	方法	结果	测试数据	实际结果
TC-REWARD-001	有效等价类	通过	获取奖品列表	奖品列表状态码 200，数量 2。
TC-REWARD-002	判定表	通过	积分不足兑换	兑换请求返回状态码 400，有成功或积分不足提示。
TC-REWARD-003	状态迁移	通过	查询我的兑换	我的兑换接口状态码 200，验证不再 500。
TC-REWARD-004	判定表	通过	未登录访问商城兑换	未登录兑换返回状态码 401。
TC-REWARD-005	UI 回归	通过	积分商城刷新页面	Selenium/Chrome 截图显示积分商城刷新不失败。
TC-NOTICE-001	UI 回归	通过	点击通知按钮	通知面板 hidden=false 且 display=flex。
TC-NOTICE-002	有效等价类	通过	通知列表	通知列表接口状态码 200。
TC-NOTICE-003	有效等价类	通过	未读数	未读数接口状态码 200。
TC-NOTICE-004	状态迁移	通过	全部已读	全部已读接口状态码 200。
TC-NOTICE-005	移动端 UI	通过	390px 通知面板	移动端通知面板截图通过。
TC-FOLLOW-001	有效等价类	通过	关注帖子作者	关注作者接口状态码 200。
TC-FOLLOW-002	状态迁移	通过	取消关注帖子作者	取消关注接口状态码 200。
TC-FOLLOW-003	判定表	通过	未登录关注	未登录关注返回状态码 401。
TC-FOLLOW-004	有效等价类	通过	关注统计	关注统计接口状态码 200。
TC-FOLLOW-005	UI 验证	通过	关注/取消关注截图	Chrome 截图保留关注/取消关注验证。
TC-ADMIN-001	有效等价类	通过	管理员查看奖品列表	后台奖品列表能够正常展示。
TC-ADMIN-002	有效等价类	通过	新增奖品：名称、所需积分、库存	新增奖品后列表可查看新记录。
TC-ADMIN-003	边界值	通过	积分为 0 或库存为 0	边界数据能够被校验，不写入无效奖品。
TC-ADMIN-004	判定表	通过	普通用户访问后台	非管理员访问被权限控制拦截。

编号	方法	结果	测试数据	实际结果
TC-ADMIN-005	状态迁移	通过	新增后刷新页面	刷新后奖品列表仍显示新增记录。

七、 测试环境与浏览器执行选择

- 操作系统：Windows。
- 后端：Spring Boot 便携运行环境。
- 前端：浏览器访问论坛页面。
- 浏览器：Google Chrome，桌面端与移动端视口各执行代表路径。
- 自动化：Selenium WebDriver + JUnit 5，并配合接口检查。

Chrome 用于积分商城、通知面板、关注按钮和后台奖品列表复核；通知中心和积分商城使用 Selenium WebDriver + JUnit 做重点回归。IE 或 Edge IE 模式不纳入本轮通过率统计。

八、 测试执行过程与截图

本节选取本人负责范围内的代表页面，说明实际执行过程和断言点。截图用于展示测试路径、页面状态和人工复核位置。

8.1 截图流程 1：积分商城奖品列表

积分商城截图用于检查奖品卡片、积分数和兑换按钮。本人主要看页面刷新后商城是否仍然能正常显示。



图 **1**　积分商城奖品列表

8.2 截图流程 2：积分兑换结果区域

兑换结果区域用于对应积分不足或兑换成功后的页面反馈。本人主要看提示文字是否清楚，以及右侧记录区域是否还能正常显示。

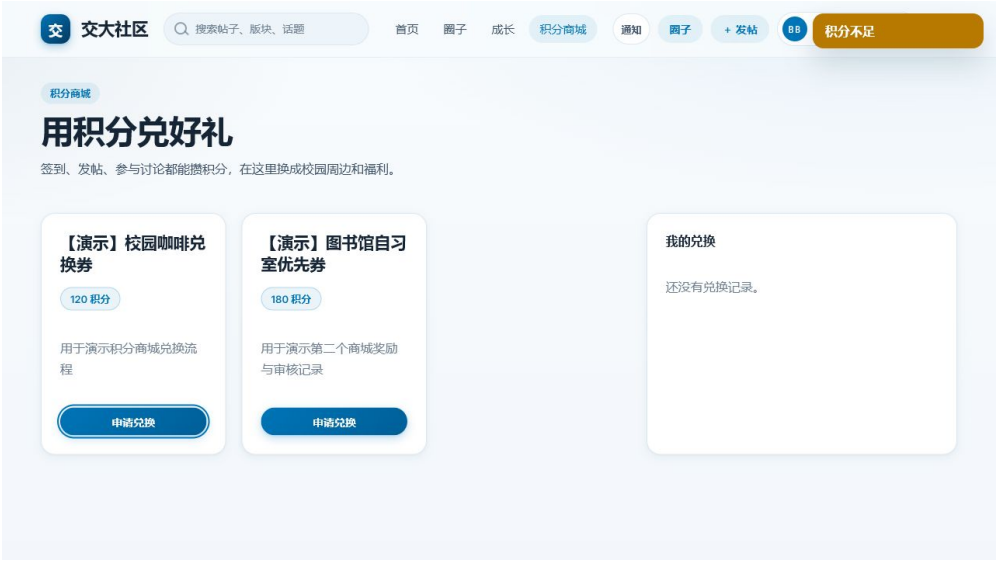


图 2　积分兑换结果区域

8.3 截图流程 3：通知中心展开状态

通知中心截图用于观察点击通知入口后的展开状态。这个页面的重点是用户能不能真正看到通知内容。



图 3　通知中心展开状态

8.4 截图流程 4：关注前的帖子详情页

关注前截图用于记录作者区域的初始按钮状态。后续关注成功或取消关注时，需要和这个状态进行对比。

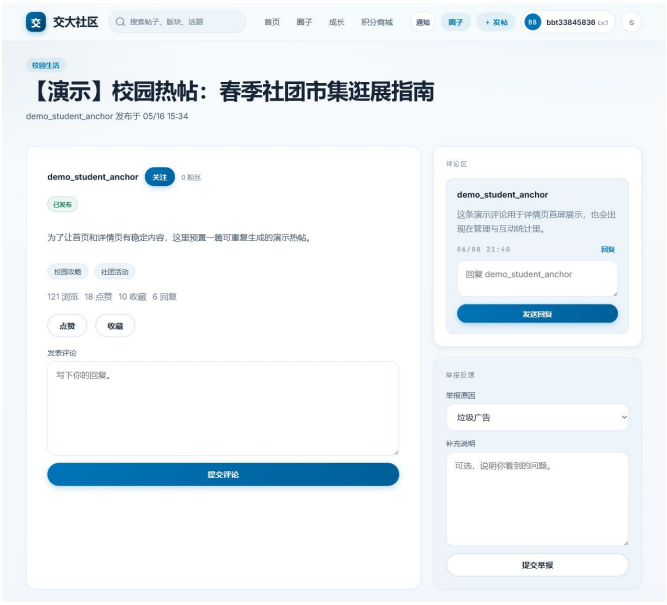


图 **4**　关注前的帖子详情页

8.5 截图流程 5：关注用户后的详情页

关注截图用于检查关注按钮状态。本人会先关注再取消，确认状态能够来回切换。

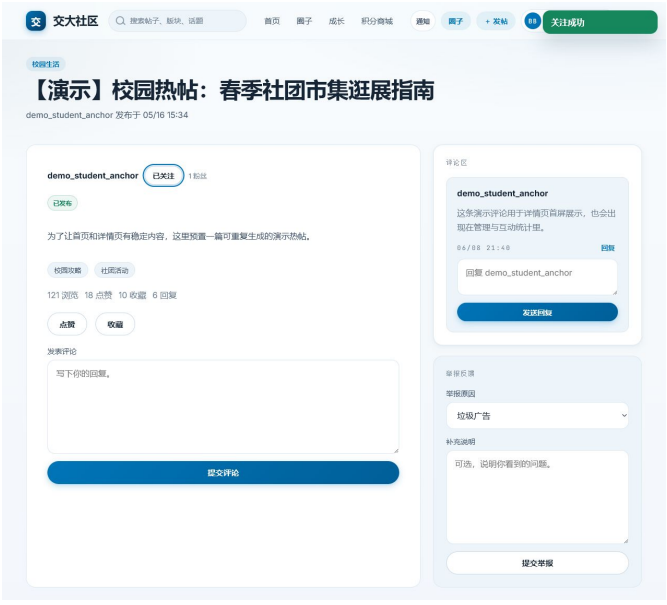


图 **5**　关注用户后的详情页

8.6 截图流程 6：取消关注后的详情页

取消关注截图用于确认关系状态能够回退。这个截图对应状态迁移用例，不只检查首次关注。

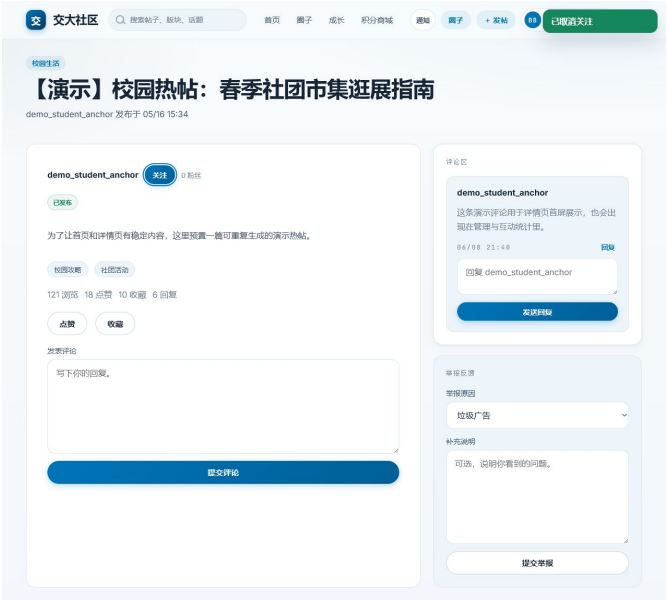


图 6　取消关注后的详情页

8.7 截图流程 7：管理员账号登录状态

管理员登录截图用于说明进入后台功能前的账号状态。后台奖品管理必须在管理员登录后执行，普通用户路径只用于权限检查。



图 7　管理员账号登录状态

8.8 截图流程 8：后台奖品管理列表

后台奖品管理截图用于说明管理员侧列表。本人主要看奖品名称、积分、库存和操作入口是否显示完整。



图 8　后台奖品管理列表

8.9 截图流程 9：后台新增奖品结果

后台新增奖品结果用于对应管理员新增奖品用例。本人主要看新增后页面是否给出明确反馈。



图 9　后台新增奖品结果

8.10 截图流程 10：后台刷新后的奖品列表

后台刷新后的奖品列表用于检查新增记录是否持久保留，而不是只停留在前端临时状态。



图 **10**　后台刷新后的奖品列表

九、 脚本开发、执行与调试

脚本调试时主要处理通知面板可见性和后台页面刷新问题。通知面板不能只判断 DOM 是否存在，还要读取 hidden、display 和 aria-expanded；后台奖品管理新增后需要刷新列表再断言，避免只验证到临时前端状态。

在 JUnit 中执行脚本时，先启动浏览器并完成登录，再进入对应页面执行等待、点击、输入和断言。调试时主要通过测试方法名、断言消息和页面文本定位问题；如果页面刷新导致元素失效，则重新查找元素后再判断状态。

脚本开发中还需要注意测试数据的可重复性。对于会改变状态的功能，脚本不能只依赖一次性的页面提示，而要在操作后重新查询页面或接口状态；对于页面异步渲染的功能，脚本需要显式等待关键元素出现，避免因为加载时间差导致误判。本人在调试时主要使用逐步缩小断言范围的方法：先确认页面能打开，再确认元素能定位，最后确认业务状态符合预期。

十、 测试结果分析

本次个人范围内共设计并执行 20 条用例，其中通过 20 条、失败 0 条、阻塞 0 条、不适用 0 条。本人判断结果时没有只看“页面有没有打开”，而是结合了状态码、页面

文本、按钮状态和刷新后的结果。这样做能减少误判，尤其是一些按钮点击后页面会异步刷新的场景。

本人负责范围内未出现失败用例。需要注意的是，“未失败”不等于没有风险，后续仍可在更多浏览器、弱网、并发和更复杂数据组合下继续扩展测试。

从执行结果看，本人负责模块中比较容易出问题的是状态保存和前后端显示一致性。后续回归时，仍然需要保留接口读取、页面刷新和截图复核三个步骤。

十一、 Bug 列表

本分工范围内未记录新的未关闭 Bug。已执行用例均有明确断言，后续可继续扩展兼容性和压力类测试。

十二、 体会与收获

我负责的几个功能都有比较明显的状态联动。积分商城要看奖品、积分和兑换记录；通知中心要看面板是否真的展开；关注关系要看按钮状态是否能切换；后台奖品管理又会影响商城展示。调试通知中心脚本时，我一开始只判断面板节点存在，后来发现节点存在并不代表用户能看到，所以又增加了 hidden 和 display 的判断。后台奖品管理也类似，新增后不能只看一次提示，还要刷新列表确认记录仍然存在。这次测试让我感觉到，WebUI 自动化不能只模拟点击，还要尽量接近用户真正看到的结果。

A 自动化测试脚本核心代码

Listing 1: 商城、通知、关注与后台奖品测试核心代码

```
class RewardNotificationAdminUiTest {
    WebDriver driver;
    WebDriverWait wait;
    JavascriptExecutor js;

    @BeforeEach
    void setUp() {
        driver = new ChromeDriver();
        wait = new WebDriverWait(driver, Duration.ofSeconds(10));
        js = (JavascriptExecutor) driver;
    }

@Test
void notificationPanelMustBecomeVisible() {
    loginAsAdmin();
    driver.get(BASE_URL + HOME_PAGE);
    click(By.cssSelector("[data-notif-toggle]"));
    WebElement panel = wait.until(ExpectedConditions.presenceOfElementLocated(By.cssSelector("[data-notif-panel]")));
    assertFalse((Boolean) js.executeScript("return arguments[0].hidden;", panel));
}
```

```
@Test
void rewardsPageShouldLoad() {
    loginAsUser();
    driver.get(BASE_URL + REWARDS_PAGE);
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("[data-reward-card]")));
    assertTrue(driver.getPageSource().contains("积分"));
}

@Test
void followButtonShouldToggleState() {
    loginAsUser();
    openFirstPostDetail();
    WebElement follow = wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-follow]")));
    follow.click();
    wait.until(ExpectedConditions.attributeContains(By.cssSelector("[data-follow]"), "aria-pressed", "true"));
}

@Test
void adminRewardListMustLoad() {
    loginAsAdmin();
    driver.get(BASE_URL + ADMIN_REWARD_PAGE);
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("[data-reward-row]")));
    assertTrue(driver.findElements(By.cssSelector("[data-reward-row]")).size() > 0);
}
}
```